

DEVOPS MASTER COURSE

Volume 14D

Model Serving, RAG, Agents and LLMOps

Inference | KServe | Ray Serve | Gateways | Prompts | RAG | Evaluation | Safety

Chapters 31-40 | 2026 Edition

Beginner to Enterprise Mastery

About This Part

Part D moves models into production through serving, gateways, prompt lifecycle, retrieval, evaluation, safety and LLM application operations.

2026 curriculum note: LLM quality is evaluated with reproducible datasets, traces and task-specific scorers. Provider output is never treated as automatically correct, safe or grounded.

Volume 14 Roadmap

Part	Coverage	Status
14A	DataOps and modern data-platform foundations	Course sequence
14B	MLOps lifecycle, registry and monitoring	Course sequence
14C	Kubeflow, Ray and distributed AI infrastructure	Course sequence
14D	Model serving, RAG, agents and LLMOps	Current part
14E	AI infrastructure operations, security, risk and FinOps	Course sequence
14F	Projects, interviews and final AI platform capstone	Course sequence

Scope, Safety and Professional Practice

- Use disposable sandboxes, local simulations or approved training subscriptions. Configure budgets, quotas and expiry before creating resources.
- Never use production identities, customer data, employer account structures, internal addresses or confidential contracts in course artifacts.
- Open-source projects, cloud services, AI models, pricing, limits and certification objectives change. Verify official documentation before production use or exam registration.
- Run failure, security, red-team and recovery tests only in approved nonproduction environments with explicit stop conditions.
- Destroy resources, revoke credentials, remove local state and sanitize every portfolio artifact.

Learning Outcomes

- Explain and design the capabilities covered by Model Serving, RAG, Agents and LLMOps.
- Implement or simulate production workflows through version-controlled automation.
- Test identity, policy, security, reliability, cost and lifecycle behavior.
- Troubleshoot complex failures using a structured evidence-first method.
- Produce sanitized architecture, laboratory, runbook and interview portfolio artifacts.

Table of Contents

Chapter 31 Online, Batch and Streaming Inference Architecture

Chapter 32 KServe Model Serving

Chapter 33 Ray Serve for Composable AI Applications

Chapter 34 Model Gateways, Routing and Provider Abstraction

Chapter 35 Prompt Management and Versioning

Chapter 36 Retrieval-Augmented Generation Architecture

Chapter 37 Vector Data and Retrieval Operations

Chapter 38 LLM and Agent Evaluation

Chapter 39 AI Guardrails, Safety and Human Oversight

Chapter 40 Enterprise LLMOps Capstone

Serving and LLMOps Engineering Reference

Completion Checklist

Additional Portfolio Projects

Practice Plan

Glossary

Official Documentation Index

Chapter 31: Online, Batch and Streaming Inference Architecture

Select serving modes based on latency, throughput, freshness, cost and failure behavior.

Why this chapter matters: Online, Batch and Streaming Inference Architecture matters because enterprise outcomes depend on more than a successful tool installation. The team must make select serving modes based on latency, throughput, freshness, cost and failure behavior. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of online, batch and streaming inference architecture.
- Design the capability using REST/gRPC, batch jobs, stream processors with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: design an inference architecture for three workloads.

Core Concepts

Select serving modes based on latency and problem framing

Select serving modes based on latency and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For online, batch and streaming inference architecture, record assumptions and the evidence that would disprove them.

throughput architecture and operating boundaries

The architecture for throughput architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

freshness automation, policy and evidence

Automation for freshness automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

cost and failure behavior. failure handling and continuous improvement

Continuous improvement for cost and failure behavior. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	REST/gRPC	Implement the core capability and define its service contract.
Automation and integration	batch jobs	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	stream processors	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	REST/gRPC	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	batch jobs	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	stream processors	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: online_batch_and_streaming_infer
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 31

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 31: Design an inference architecture for three workloads

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for online, batch and streaming inference architecture.
2. Create the smallest safe implementation or detailed simulation using REST/gRPC, batch jobs, stream processors.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Online, Batch and Streaming Inference Architecture?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Online, Batch and Streaming Inference Architecture is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 32: KServe Model Serving

Deploy autoscaled inference services with revisions, traffic controls and observability.

Why this chapter matters: KServe Model Serving matters because enterprise outcomes depend on more than a successful tool installation. The team must make deploy autoscaled inference services with revisions, traffic controls and observability. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of kserve model serving.
- Design the capability using KServe, Knative, Kubernetes with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: deploy a canary inference service.

Core Concepts

Deploy autoscaled inference services with revisions and problem framing

Deploy autoscaled inference services with revisions and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For kserve model serving, record assumptions and the evidence that would disprove them.

traffic controls and observability. architecture and operating boundaries

The architecture for traffic controls and observability. architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

KServe Model Serving automation, policy and evidence

Automation for kserve model serving automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

KServe Model Serving failure handling and continuous improvement

Continuous improvement for kserve model serving failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	KServe	Implement the core capability and define its service contract.
Automation and integration	Knative	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	Kubernetes	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	KServe	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	Knative	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	Kubernetes	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: kserve_model_serving
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 32

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 32: Deploy a canary inference service

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for kserve model serving.
2. Create the smallest safe implementation or detailed simulation using KServe, Knative, Kubernetes.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design KServe Model Serving?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- KServe Model Serving is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 33: Ray Serve for Composable AI Applications

Build scalable multi-model and Python-native inference graphs.

Why this chapter matters: Ray Serve for Composable AI Applications matters because enterprise outcomes depend on more than a successful tool installation. The team must make build scalable multi-model and python-native inference graphs. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of ray serve for composable ai applications.
- Design the capability using Ray Serve, KubeRay, autoscaling with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: deploy a multi-stage ray serve application.

Core Concepts

Build scalable multi-model and Python-native inference graphs. and problem framing

Build scalable multi-model and Python-native inference graphs. and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For ray serve for composable ai applications, record assumptions and the evidence that would disprove them.

Ray Serve for Composable AI Applications architecture and operating boundaries

The architecture for ray serve for composable ai applications architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

Ray Serve for Composable AI Applications automation, policy and evidence

Automation for ray serve for composable ai applications automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Ray Serve for Composable AI Applications failure handling and continuous improvement

Continuous improvement for ray serve for composable ai applications failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Ray Serve	Implement the core capability and define its service contract.
Automation and integration	KubeRay	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	autoscaling	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Ray Serve	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	KubeRay	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	autoscaling	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: ray_serve_for_composable_ai_appl
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 33

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 33: Deploy a multi-stage Ray Serve application

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for ray serve for composable ai applications.
2. Create the smallest safe implementation or detailed simulation using Ray Serve, KubeRay, autoscaling.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Ray Serve for Composable AI Applications?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Ray Serve for Composable AI Applications is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 34: Model Gateways, Routing and Provider Abstraction

Centralize authentication, quotas, routing, fallback and cost without hiding critical semantics.

Why this chapter matters: Model Gateways, Routing and Provider Abstraction matters because enterprise outcomes depend on more than a successful tool installation. The team must make centralize authentication, quotas, routing, fallback and cost without hiding critical semantics. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of model gateways, routing and provider abstraction.
- Design the capability using AI gateways, rate limiting, policy with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: design a multi-provider model gateway.

Core Concepts

Centralize authentication and problem framing

Centralize authentication and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For model gateways, routing and provider abstraction, record assumptions and the evidence that would disprove them.

quotas architecture and operating boundaries

The architecture for quotas architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

routing automation, policy and evidence

Automation for routing automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

fallback and cost without hiding critical semantics. failure handling and continuous improvement

Continuous improvement for fallback and cost without hiding critical semantics. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	AI gateways	Implement the core capability and define its service contract.
Automation and integration	rate limiting	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	policy	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	AI gateways	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	rate limiting	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	policy	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: model_gateways_routing_and_provi
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 34

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 34: Design a multi-provider model gateway

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for model gateways, routing and provider abstraction.
2. Create the smallest safe implementation or detailed simulation using AI gateways, rate limiting, policy.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Model Gateways, Routing and Provider Abstraction?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Model Gateways, Routing and Provider Abstraction is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 35: Prompt Management and Versioning

Treat prompts, tools, templates and system instructions as tested release artifacts.

Why this chapter matters: Prompt Management and Versioning matters because enterprise outcomes depend on more than a successful tool installation. The team must make treat prompts, tools, templates and system instructions as tested release artifacts. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of prompt management and versioning.
- Design the capability using MLflow prompt management, Git, registries with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: create a prompt release and rollback workflow.

Core Concepts

Treat prompts and problem framing

Treat prompts and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For prompt management and versioning, record assumptions and the evidence that would disprove them.

tools architecture and operating boundaries

The architecture for tools architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

templates and system instructions as tested release artifacts. automation, policy and evidence

Automation for templates and system instructions as tested release artifacts. automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Prompt Management and Versioning failure handling and continuous improvement

Continuous improvement for prompt management and versioning failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	MLflow prompt management	Implement the core capability and define its service contract.
Automation and integration	Git	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	registries	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	MLflow prompt management	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	Git	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	registries	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: prompt_management_and_versioning
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 35

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 35: Create a prompt release and rollback workflow

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for prompt management and versioning.
2. Create the smallest safe implementation or detailed simulation using MLflow prompt management, Git, registries.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Prompt Management and Versioning?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Prompt Management and Versioning is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 36: Retrieval-Augmented Generation Architecture

Engineer ingestion, chunking, embeddings, retrieval, reranking, citations and freshness.

Why this chapter matters: Retrieval-Augmented Generation Architecture matters because enterprise outcomes depend on more than a successful tool installation. The team must make engineer ingestion, chunking, embeddings, retrieval, reranking, citations and freshness. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of retrieval-augmented generation architecture.
- Design the capability using Vector databases, embedding models, pipelines with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: build a small rag system with evaluation data.

Core Concepts

Engineer ingestion and problem framing

Engineer ingestion and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For retrieval-augmented generation architecture, record assumptions and the evidence that would disprove them.

chunking architecture and operating boundaries

The architecture for chunking architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

embeddings automation, policy and evidence

Automation for embeddings automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

retrieval failure handling and continuous improvement

Continuous improvement for retrieval failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Vector databases	Implement the core capability and define its service contract.
Automation and integration	embedding models	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	pipelines	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Vector databases	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	embedding models	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	pipelines	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: retrieval_augmented_generation_a
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 36

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 36: Build a small RAG system with evaluation data

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for retrieval-augmented generation architecture.
2. Create the smallest safe implementation or detailed simulation using Vector databases, embedding models, pipelines.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Retrieval-Augmented Generation Architecture?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Retrieval-Augmented Generation Architecture is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 37: Vector Data and Retrieval Operations

Manage indexes, metadata filters, consistency, privacy, backup and re-embedding.

Why this chapter matters: Vector Data and Retrieval Operations matters because enterprise outcomes depend on more than a successful tool installation. The team must manage indexes, metadata filters, consistency, privacy, backup and re-embedding. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of vector data and retrieval operations.
- Design the capability using pgvector, Milvus, OpenSearch, Qdrant with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: operate a versioned vector index.

Core Concepts

Manage indexes and problem framing

Manage indexes and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For vector data and retrieval operations, record assumptions and the evidence that would disprove them.

metadata filters architecture and operating boundaries

The architecture for metadata filters architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

consistency automation, policy and evidence

Automation for consistency automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

privacy failure handling and continuous improvement

Continuous improvement for privacy failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	pgvector	Implement the core capability and define its service contract.
Automation and integration	Milvus	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	OpenSearch	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Qdrant	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	pgvector	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	Milvus	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```
capability: vector_data_and_retrieval_operat
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 37
```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 37: Operate a versioned vector index

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for vector data and retrieval operations.
2. Create the smallest safe implementation or detailed simulation using pgvector, Milvus, OpenSearch, Qdrant.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Vector Data and Retrieval Operations?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Vector Data and Retrieval Operations is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 38: LLM and Agent Evaluation

Measure task success, groundedness, safety, latency and cost using reproducible datasets.

Why this chapter matters: LLM and Agent Evaluation matters because enterprise outcomes depend on more than a successful tool installation. The team must measure task success, groundedness, safety, latency and cost using reproducible datasets. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of LLM and agent evaluation.
- Design the capability using MLflow GenAI evaluation, scorers, traces with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: build an LLM evaluation suite and release gate.

Core Concepts

Measure task success and problem framing

Measure task success and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For LLM and agent evaluation, record assumptions and the evidence that would disprove them.

groundedness architecture and operating boundaries

The architecture for groundedness architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

safety automation, policy and evidence

Automation for safety automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

latency and cost using reproducible datasets. failure handling and continuous improvement

Continuous improvement for latency and cost using reproducible datasets. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	MLflow GenAI evaluation	Implement the core capability and define its service contract.
Automation and integration	scorers	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	traces	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	MLflow GenAI evaluation	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	scorers	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	traces	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: llm_and_agent_evaluation
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 38

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 38: Build an LLM evaluation suite and release gate

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for LLM and agent evaluation.
2. Create the smallest safe implementation or detailed simulation using MLflow GenAI evaluation, scorers, traces.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design LLM and Agent Evaluation?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- LLM and Agent Evaluation is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 39: AI Guardrails, Safety and Human Oversight

Apply input/output controls, authorization, tool limits, red teaming and escalation.

Why this chapter matters: AI Guardrails, Safety and Human Oversight matters because enterprise outcomes depend on more than a successful tool installation. The team must make apply input/output controls, authorization, tool limits, red teaming and escalation. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of ai guardrails, safety and human oversight.
- Design the capability using Policy engines, content filters, sandboxing with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: threat-model and red-team an ai workflow.

Core Concepts

Apply input/output controls and problem framing

Apply input/output controls and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For ai guardrails, safety and human oversight, record assumptions and the evidence that would disprove them.

authorization architecture and operating boundaries

The architecture for authorization architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

tool limits automation, policy and evidence

Automation for tool limits automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

red teaming and escalation. failure handling and continuous improvement

Continuous improvement for red teaming and escalation. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Policy engines	Implement the core capability and define its service contract.
Automation and integration	content filters	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	sandboxing	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Policy engines	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	content filters	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	sandboxing	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: ai_guardrails_safety_and_human_o
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 39

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 39: Threat-model and red-team an AI workflow

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for ai guardrails, safety and human oversight.
2. Create the smallest safe implementation or detailed simulation using Policy engines, content filters, sandboxing.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design AI Guardrails, Safety and Human Oversight?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- AI Guardrails, Safety and Human Oversight is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 40: Enterprise LLMOps Capstone

Integrate prompts, RAG, gateway, evaluation, tracing, safety, deployment and cost controls.

Why this chapter matters: Enterprise LLMOps Capstone matters because enterprise outcomes depend on more than a successful tool installation. The team must make integrate prompts, rag, gateway, evaluation, tracing, safety, deployment and cost controls. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of enterprise llmops capstone.
- Design the capability using MLflow, KServe/Ray Serve, OpenTelemetry with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: deliver a production-ready llm application platform.

Core Concepts

Integrate prompts and problem framing

Integrate prompts and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For enterprise llmops capstone, record assumptions and the evidence that would disprove them.

RAG architecture and operating boundaries

The architecture for rag architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

gateway automation, policy and evidence

Automation for gateway automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

evaluation failure handling and continuous improvement

Continuous improvement for evaluation failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	MLflow	Implement the core capability and define its service contract.
Automation and integration	KServe/Ray Serve	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	OpenTelemetry	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	MLflow	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	KServe/Ray Serve	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	OpenTelemetry	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: enterprise_llmops_capstone
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 14D
chapter: 40

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 40: Deliver a production-ready LLM application platform

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for enterprise llmops capstone.
2. Create the smallest safe implementation or detailed simulation using MLflow, KServe/Ray Serve, OpenTelemetry.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Enterprise LLMOps Capstone?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Enterprise LLMOps Capstone is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Serving and LLMOps Engineering Reference

Use this reference to review product outcomes, architecture boundaries, operating controls and evidence required for production approval.

Domain	Required outcome	Validation evidence
Product and users	Named consumers, outcome hypothesis, roadmap and lifecycle owner	Research summary, service contract and adoption data
Architecture	Control, execution, data and evidence boundaries with bounded blast radius	Diagrams, ADRs, dependency and failure tests
Identity and security	Least privilege, keyless workloads, protected administration and auditable exceptions	Effective-access, denied-action and audit evidence
Delivery and policy	Versioned automation, policy, promotion, rollback and provenance	Pipeline, policy and artifact verification results
Reliability and operations	SLOs, telemetry, capacity, incident response, recovery and deprecation	Dashboards, game-day timeline and restore evidence
Data and AI governance	Ownership, classification, quality, privacy, lineage and responsible use	Catalog, tests, risk and approval records
Economics and value	Allocated cost, unit economics, investment rationale and optimization safety	Cost model, outcome metrics and decision log
People and governance	Clear decisions, support, skills, change plan and feedback loops	RACI, operating cadence, survey and action backlog

Completion Checklist

- [] I can explain every chapter in Volume 14D using architecture, operations and business language.
- [] I can implement or simulate at least one complete service with identity, policy, telemetry and lifecycle controls.
- [] I can demonstrate expected, denied and failure behavior with time-stamped evidence.
- [] I can discuss security, reliability, cost, adoption and organizational trade-offs.
- [] I can produce a sanitized portfolio artifact and defend the design in an interview or review board.

Additional Portfolio Projects

1. Service catalog and ownership quality automation.
2. Keyless CI/CD and workload-identity reference implementation.
3. Policy promotion pipeline with expiring exceptions.
4. Golden path or pipeline template with user-journey telemetry.
5. SLO dashboard and bounded failure game day.
6. Cost allocation and unit-economics review.
7. Security threat model and evidence-based remediation plan.
8. Backup, restore, failover and failback exercise.
9. Migration or adoption factory for three fictional teams.
10. Executive architecture or investment memo with measured outcomes.

Practice Plan

Days	Focus	Required output
1-5	Concepts, users and current-state discovery	Research notes, value stream and capability map
6-10	Architecture and service contracts	Diagrams, API/schema and ADRs
11-16	Implementation and automation	Working sandbox or detailed simulation
17-21	Security, policy and governance	Threat model, policy tests and exceptions
22-26	Reliability, telemetry and recovery	SLOs, dashboards and failure drill
27-31	Cost, capacity and lifecycle	Unit cost, quota and deprecation plan
32-36	Adoption, people and operating model	RACI, support and change plan
37-41	Troubleshooting and interview practice	Timed investigation and interview notes
42-45	Capstone and portfolio	Final defense, cleanup and sanitized evidence pack

Glossary

Term	Definition
Control plane	The APIs and controllers that accept desired state and coordinate execution.
Developer portal	A user interface that aggregates catalog, documentation, workflows and operational context.
Internal developer platform	The integrated products, APIs, automation and processes that support software delivery.
Golden path	An opinionated, supported default journey for a common engineering task.
Data product	An owned data asset or interface with consumers, quality, documentation and lifecycle.
MLOps	Practices and systems for reproducible, governed delivery and operation of machine-learning models.
LLMOps	Lifecycle, evaluation, deployment, observability, safety and cost practices for LLM applications.
Sociotechnical system	A system whose outcomes emerge from interactions among people, process, incentives and technology.
Unit economics	Cost and value measured per meaningful business or technical output.
Error budget	The allowed unreliability implied by a service-level objective.
Policy as code	Machine-testable policy stored, reviewed and promoted through version control.
Evidence pack	Sanitized architecture, configuration, test, telemetry and decision records used to prove readiness.

Official Documentation Index

The following primary or official sources were consulted for this 2026 edition. Verify current versions, service behavior, prices, limits and certification objectives before use.

- [MLflow Documentation](#)
- [MLflow Tracking](#)
- [MLflow Model Registry](#)
- [Kubeflow Introduction](#)

- [Kubeflow Pipelines](#)
- [Kubeflow Trainer](#)
- [KServe Documentation](#)
- [Ray Documentation](#)
- [Ray Serve](#)
- [NIST AI Risk Management Framework](#)

Volume 14D completion: preserve your architecture, code or simulation, test evidence, recovery timeline, cost/value review and sanitized final report as one portfolio package.